

Audio Spectrogram Transformer for Synthetic Speech Detection via Speech Formant Analysis

Luca Cuccovillo , Milica Gerhardt , and Patrick Aichroth 
Fraunhofer Institute for Digital Media Technology IDMT
Ehrenbergstraße 31, 98693 Ilmenau, Germany
{luca.cuccovillo, milica.gerhardt, patrick.aichroth}@idmt.fraunhofer.de

Abstract—In this paper, we address the challenge of synthetic speech detection, which has become increasingly important due to the latest advancements in text-to-speech and voice conversion technologies. We propose a novel multi-task neural network architecture, designed to be interpretable and specifically tailored for audio signals. The architecture includes a feature bottleneck, used to autoencode the input spectrogram, predict the fundamental frequency (f_0) trajectory, and classify the speech as synthetic or natural. Hence, the synthesis detection can be considered a byproduct of attending to the energy distribution among vocal formants, providing a clear understanding of which characteristics of the input signal influence the final outcome. Our evaluation on the ASVspoof 2019 LA partition indicates better performance than the current state of the art, with an AUC score of 0.900.

Index Terms—synthetic speech detection, audio deepfakes, spectrogram transformer, voice formants

I. INTRODUCTION

Recent progress in Text-To-Speech (TTS) and Voice Conversion (VC) technologies has achieved an unparalleled quality, making it increasingly challenging for human listeners to differentiate between synthetic and natural voices [1]. This development has raised concerns about the potential misuse of synthetic speech, as highlighted e.g. in a recent report by Europol [2]. As a result, research efforts have focused on the topic, and especially on exploring end-to-end, data-driven solutions based on Artificial Intelligence (AI), driven primarily by the ASVspoof community [3].

However, it can be argued that the urgency of this task has led to the creation of datasets that have several problems [4]. These problems include flawed data distribution, which in turn also affects the performance of models trained with these data. Müller *et al.* [5], for example, found an uneven silence distribution in the training and testing portions of the ASVspoof 2019 LA dataset, showing that the Equal Error Rate (EER) of a widely used anti-spoofing model RawNet2 [6] rises sharply from 3.6% to 15.5% after removing leading and trailing silence from the dataset. In another study, Müller *et al.* [7] showed that the performance of several architectures trained on ASVspoof, although very accurate on the competition test dataset, was no better than random guessing for out-of-distribution data that originated from the Internet.

This paper was supported by the BMBF SpeechTrust+ project (grant no 13N16267), and by the EU Horizon Europe vera.ai project (grant no 101070093).

Another fundamental problem contributing to the poor recognition performance of existing approaches to speech synthesis recognition is the lack of interpretability in their end-to-end architectures. In the field of image processing the challenge of interpretability was recognized early on, leading e.g. to the development of methods that identify relevant regions in the input image, such as Grad-CAM [8]. However, the application of such approaches to audio spectrograms has limited effectiveness. Thus, there is a need for speech synthesis detection approaches that are specifically designed to operate in an interpretable manner, while taking the characteristics of audio signals into account.

In this work, we propose to approach the problem of synthetic speech recognition from a different perspective, by training a neural network that focuses primarily on other tasks, and for which the detection of synthetic speech is essentially a byproduct. The network design is based on the hypothesis that the energy distribution among vocal formants of synthetic speech exhibits anomalies. Therefore, the architecture consists of a multi-task network with a feature bottleneck, used to autoencode the input spectrogram, then to predict the trajectory of the fundamental frequency (f_0), and finally to determine whether the speech is synthetic or not.

The network relies on sequence-to-sequence (seq2seq) transformers as basic building blocks, exploiting their inherent attention mechanism [9]: Attention is well suited for f_0 prediction, due to its strong correlation with other speech formants, as well as for autoencoding, since speech formants are crucial for minimizing the reconstruction error. In that sense, speech synthesis detection becomes a byproduct of attention to the energy distribution among vocal formants, and the approach provides a clear understanding about which characteristics of the input signal contribute to the final outcome.

The paper is organized as follows: Section II describes the details of the architecture, providing example outputs of the respective building blocks; Section III outlines the evaluation setup used in our experiments, which are subsequently described in Section IV. Finally, Section V concludes with a summary and outlook for future research based on this work.

II. PROPOSED ARCHITECTURE

Our architecture consists of four basic components, summarized in Figure 1 and detailed in the following sections:

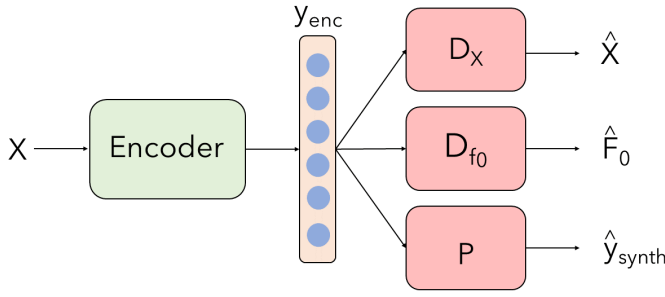


Fig. 1. Architecture of the proposed Speech Formant Analysis Transformer Network (SFAT-Net)

- 1) Spectrogram Encoder E : A seq2seq transformer that converts a log-spectrogram of an input file into a suitable sequence of embeddings y_{enc} , acting as robust feature bottleneck (Section II-A).
- 2) Spectrogram Decoder D_X : A seq2seq transformer that converts the embeddings y_{enc} into an approximation of the original input log-spectrogram (Section II-B).
- 3) Fundamental Frequency Decoder D_{f_0} : A seq2seq transformer that converts the embeddings y_{enc} into an approximation of the f_0 trajectory of the input speech (Section II-C).
- 4) Synthesis Predictor P : A seq2seq transformer that converts the embeddings y_{enc} into a 2-dimensional vector signaling the presence of synthetic speech (Section II-D).

A. Spectrogram Encoder

Let us denote with $X \in \mathbb{R}^{L \times M}$ the log-spectrogram of an input recording $x(t)$, with L denoting the number of frames and M the number of frequency bins of X .

On a high level, the spectrogram encoder E is a function that maps the log-spectrogram into a sequence of embeddings y_{enc} , i.e.:

$$y_{\text{enc}} = E(X, \Theta_{\text{enc}}), \quad (1)$$

in which Θ_{enc} denotes the set of model parameters.

In particular, we decided to realize E in a similar way as encoders of vision transformers (ViTs) [10].

In a first step, the input log-spectrogram X is split into a sequence of non-overlapping 2D patches $x_p \in \mathbb{R}^{P_L \times P_M}$. Each patch has a fixed amount of rows (P_L) and of columns (P_M), and the total amount of patches N is fixed and equal to $(L \cdot M)/(P_L \cdot P_M)$:

$$x_p = \text{reshape}(X, P_L, P_M), \quad (2)$$

with $p \in 1 \dots N$.

Each patch x_p is then flattened and projected with a learnable linear layer to have a fixed dimension D :

$$z_p = W_{(P_L P_M \rightarrow D)} \cdot \text{flatten}(x_p), \quad (3)$$

where $W_{(P_L P_M \rightarrow D)} \in \mathbb{R}^{D \times (P_L \cdot P_M)}$ is a learnt projection matrix.

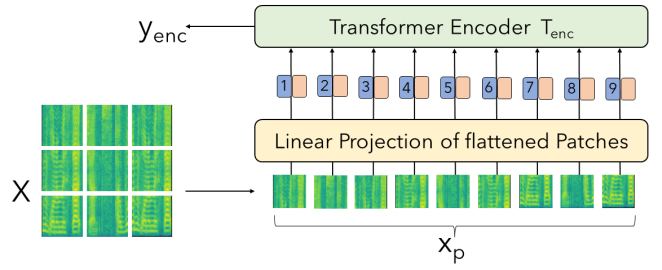


Fig. 2. Architecture of the Spectrogram Encoder

At the end of the process, the input spectrogram is therefore converted in the following sequence of patch embeddings:

$$\mathbf{z} \in \mathbb{R}^{N \times D} = [z_1, z_2, \dots, z_N], \quad (4)$$

in which the amount of patches N is also the length of the input sequence to the subsequent transformer T_{enc} , and in which D is equal to its latent space.

The final embeddings y_{enc} are computed by feeding the sequence with positional information to the transformer:

$$y_{\text{enc}} = T_{\text{enc}}(\mathbf{z} + \mathbf{z}_{\text{pos}}), \quad (5)$$

with $\mathbf{z}_{\text{pos}}$ denoting standard learnable 1D position embeddings.

Similarly to ViT [10], the transformer encoder T_{enc} consists of a sequence of alternating layers of multi-head self-attention (MSA) and multi-layer perceptron (MLP) blocks. Layer normalization is applied before every block, and residual connections after each block.

B. Spectrogram Decoder

On a high level, the spectrogram decoder D_X is a function that maps the sequence of embeddings $y_{\text{enc}} \in \mathbb{R}^{N \times D}$ to a matrix $\hat{X} \in \mathbb{R}^{L \times F}$, which is approximately identical to spectrogram X used as input to the encoder:

$$D_X(y_{\text{enc}}, \Theta_{\text{dec}}^X) = \hat{X} \approx X, \quad (6)$$

in which Θ_{dec}^X denotes the set of model parameters.

Let us note with $y_p \in \mathbb{R}^{N \times D}$ the p -th element of the encoder output sequence. In a first step we project y_{enc} to a new embedding space in $N \times D^X$ dimensions:

$$\mathbf{z}^X = \{W_{(D \rightarrow D^X)} \cdot y_{\text{enc}}\}, \quad (7)$$

where $W_{(D \rightarrow D^X)} \in \mathbb{R}^{D^X \times D}$ is a learnt projection matrix.

We then feed this sequence to a decoding transformer T^X to become a sequence of patch embeddings:

$$\hat{\mathbf{z}}^X = T^X(\mathbf{z}^X + \mathbf{z}_{\text{pos}}^X), \quad (8)$$

where $\mathbf{z}_{\text{pos}}^X$ denotes a new set of 1D positional embeddings.

Let us denote with $\hat{z}_p^X \in \mathbb{R}^{D^X}$ the p -th element of $\hat{\mathbf{z}}^X$. Each of these N vectors can be mapped to a specific patch of the input spectrogram space by inverting the projection in eq. (3) and the reshaping in eq. (2).

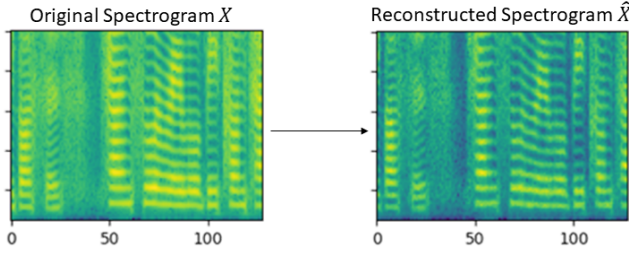


Fig. 3. Example spectrogram reconstruction

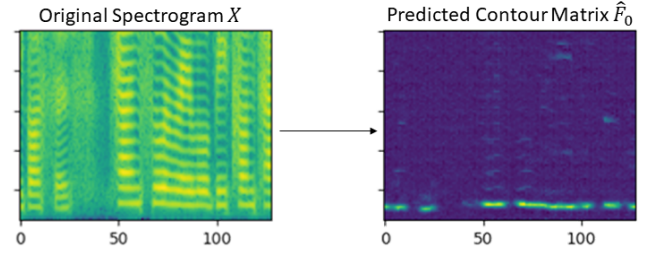


Fig. 4. Example f_0 contour matrix prediction

Accordingly, we obtain a new vector $\hat{x}_p^X \in \mathbb{R}^{P_L P_M}$:

$$\hat{x}_p^X = W_{(D^X \rightarrow P_L P_M)} \cdot \hat{z}_p^X, \quad (9)$$

and then we invert the initial reshaping

$$\hat{X} = \text{reshape}^{-1}(x_p^X, P_L, P_M), \quad (10)$$

resulting in an output matrix $\hat{X} \in \mathbb{R}^{L \times F}$.

In order to obtain an output $\hat{X}$ as similar as possible to input spectrogram X , at training time we minimize an associated autoencoding loss:

$$l_{\text{auto}}(X | \Theta_{\text{enc}}, \Theta_{\text{dec}}^X) = \left\| X - D_X(E(X, \Theta_{\text{enc}}), \Theta_{\text{dec}}^X) \right\|_2, \quad (11)$$

with $\|\cdot\|_2$ denoting the l^2 norm. An example reconstructed spectrogram $\hat{X}$ obtained after minimizing this loss is depicted in Figure 3.

C. Fundamental Frequency Decoder

The goal of the f_0 decoder D_{f_0} is to track the fundamental frequency of the input speech given the sequence of encoded spectrogram embeddings y_{enc} .

Let us denote with $F_0 \in \mathbb{R}^{L \times M}$ the contour matrix of the f_0 trajectory, i.e. a matrix with the same dimension as the input spectrogram, in which

$$F_0(l, m) = \begin{cases} 1 & \text{if } f_0(m) \approx \frac{f_s}{2M} \cdot m \\ 0 & \text{otherwise} \end{cases}. \quad (12)$$

The task of the decoder can be seen as a regression problem, in which y_{enc} must be mapped to an estimate of X^{f_0} :

$$D_{f_0}(y_{\text{enc}}, \Theta_{\text{dec}}^{f_0}) = \hat{F}_0 \approx F_0, \quad (13)$$

in which $\Theta_{\text{dec}}^{f_0}$ denotes the set of model parameters.

To realize this task, we use the same architecture as for spectrogram reconstruction, and train a separate transformer T^{f_0} with the respective projection matrices $W_{(D \rightarrow D^{f_0})}$ and $W_{(D^{f_0} \rightarrow (P_L P_M))}$, and learnt positional embeddings $\mathbf{z}_{\text{pos}}^{f_0}$.

The loss of the network, however, is based on the desired contour matrix X^{f_0} :

$$l_{f_0}(X, F_0 | \Theta_{\text{enc}}, \Theta_{\text{dec}}^{f_0}) = \left\| F_0 - D_X(E(X, \Theta_{\text{enc}}), \Theta_{\text{dec}}^{f_0}) \right\|_2, \quad (14)$$

with $\|\cdot\|_2$ denoting the l^2 norm.

The most critical aspect for this task is the creation of proper ground truths for the contour matrix F_0 . We propose to do this

by using the pYin algorithm [11], which is known to be very robust in absence of additive noise.

Rather than using directly the formulation in eq. (12), we further propose to give as target a *filtered* version of F_0 , in which we apply a 3×3 Gaussian kernel with unitary variance to smooth the matrix. We made this choice on the one hand to smooth the error function and help convergence, and on the other hand to penalize small errors less severely than large ones. An example of a reconstructed f_0 contour matrix $\hat{F}_0$ obtained with this approach is depicted in Figure 3.

D. Synthesis Predictor

The goal of the synthesis Predictor P is to convert the embeddings y_{enc} into a 2-dimensional vector y signaling the presence of synthetic speech by means of one-hot encoding.

$$P(y_{\text{enc}}, \Theta_{\text{dec}}^P) = \hat{y}_{\text{synth}}, \quad (15)$$

in which Θ_{dec}^P denotes the set of model parameters, and $\hat{y}_{\text{synth}} \in \mathbb{R}^2$ realizes a one-hot encoding of the synthesis detection:

$$\arg \max(\hat{y}_{\text{synth}}) = \begin{cases} 1 & \text{formant energies seem synthetic} \\ 0 & \text{otherwise} \end{cases} \quad (16)$$

The predictor consists of a standard transformer with a class token used as input to a classification layer, which we will describe in the following.

In a first step we project y_{enc} to a new embedding space in $N \times D^P$ dimensions:

$$\mathbf{z}^P = \{W_{(D \rightarrow D^P)} \cdot y_{\text{enc}}\}, \quad (17)$$

where $W_{(D \rightarrow D^P)} \in \mathbb{R}^{D^P \times D}$ is a learnt projection matrix.

Since the goal of the predictor can be associated to a classification task, we then prepend this sequence with an additional vector $z_0 \in \mathbb{R}^{D^P}$, corresponding to the class token of ViT architectures [10].

The outcome is a sequence of $N + 1$ elements:

$$\tilde{\mathbf{z}}^P \in \mathbb{R}^{(N+1) \times D^P} = [z_0, \mathbf{z}^P], \quad (18)$$

which is processed by a prediction transformer T^P to become a sequence of patch embeddings:

$$\hat{\mathbf{z}}^P = T^P(\tilde{\mathbf{z}}^P + \mathbf{z}_{\text{pos}}^P), \quad (19)$$

where $\mathbf{z}_{\text{pos}}^P$ denotes a new set of 1D positional embeddings.

TABLE I
TRANSFORMER HYPER-PARAMETERS

Component	Global Params		MLP Blocks		Self-Attention Blocks	
	Depth	Embedding Size	Dimensions	Dropout	Number of Heads	Head Dimension
E – Spectrogram Encoder	8	1024	2048	0	8	64
D_{dec}^X – Spectrogram Decoder	6	512	1024	0	8	64
$D_{\text{dec}}^{f_0}$ – F0 Decoder	4	512	1024	0	8	64
P – Synthesis Predictor	4	512	1024	0.1	6	64

Finally, we select only the first element $\hat{z}_0^P \in \mathbb{R}^{D^P}$ of the transformed output sequence, to be processed by a classification layer yielding the final output:

$$\hat{y}_{\text{synth}} = f(W_{\text{synth}} \cdot \hat{z}_0^P + b_{\text{synth}}), \quad (20)$$

with W_{synth} and b_{synth} being weight matrix and bias vector of the linear classification layer, and $f(\cdot)$ its activation function.

In our experiments, we decided to use a linear activation function at inference phase, and a sigmoid function with Binary Cross-Entropy (BCE) loss at training phase:

$$l_P(X, y_{\text{synth}} | \Theta_{\text{enc}}, \Theta_{\text{dec}}^P) = \text{BCE}(y_{\text{synth}}, P(E(X, \Theta_{\text{enc}}), \Theta_{\text{dec}}^P)) \quad (21)$$

III. EXPERIMENTAL SETUP

In this section we introduce the dataset used in our experiments, and detail the parameters for creating and training our speech formant analysis transformer.

A. Dataset

All the experiments were performed on the ASVspoof2019 speech dataset [12], released for the homonymous challenge. This dataset contains speech utterances which the competitors are meant to analyse in order to jointly perform speech spoofing detection and automatic speaker verification.

In particular, ASVspoof includes a Logical Access (LA) set of files comprising both real and synthetic voice utterances, and is divided into three partitions called *train*, *dev* and *eval*. All partitions include natural speech signals along with synthetic speech samples generated with various methods.

The *train* and *dev* partitions have been created using the same set of 6 synthesis algorithms (A01, A02, ..., A06), while the *eval* partition includes samples generated with 13 different techniques (A07, A08, ..., A19), to simulate a open scenario with attacks performed using unknown algorithms on unknown voices.

Even though ASVspoof features an even distribution of voices, the distribution of silent intervals within it is known to be uneven [5]. In our experiments, we therefore decided to remove any leading or trailing silence by applying the MarbleNet model for voice activity detection [13], and to normalize the volume of all utterances according to the EBU recommendation on the audio signal loudness [14] using a sampling frequency of 16,000 Hz.

We merged the *train* and *dev* partitions together in a unique training set, and left the *eval* partition for tests.

Without any further pre-processing, both training and test distributions would have presented a large imbalance between natural and synthetic utterances, with ratios of 1:6 and 1:13 between the two classes, respectively. In order to partially mitigate this issue, we oversampled the natural recordings until reaching a ratio of 1:1 during training, and applied a random starting offset to each trial. This should have prevented the repetition of the same content across single epochs, but might have nevertheless created some unwanted overfitting to the recording transmission channels [4].

B. Training parameters

In Table I we report the set of training parameters related to the transformers involved in the spectrogram analysis.

We designed the encoder E to contain the majority of the parameters, having both the largest embedding size and the highest amount of alternating pairs of multi-head self-attention (MSA) and multi-layer perceptron (MLP) blocks, which hereon we will refer to as *depth*.

The spectrogram decoder D_{dec}^X , while still rich in parameters, has a slightly lower depth and an embedding dimension that is half of the size of E . To reflect a decrease in decoding complexity, we further reduced the depth of the f_0 decoder $D_{\text{dec}}^{f_0}$, while maintaining the same embedding dimension as D_{dec}^X . In order to avoid excessive reduction in depth for the synthesis predictor P , we employed a reduction in the number of heads used in the self-attention layers, and introduced a dropout of 0.1 in the MLP blocks to decrease the risk of overfitting. For all components, the amount of neurons of the MLP layers within the transformers was twice the respective embedding dimension.

Furthermore, each input audio file was processed by a standard pre-emphasis filter with coefficient 0.97 to emphasize the mid-high frequencies, and we imposed a window length of 32 ms and a hop size of 16 ms to extract the log spectrograms. The final input spectrograms were fixed to have size of $L = 128$ frames by $M = 256$ frequency bins, corresponding to 2.064 seconds of content. The patches were created by applying a $P_L = 16 \times P_M = 16$ grid, corresponding to a sequence with length $N = 128$.

Finally, the training was performed with an initial learning rate of $1e-4$, a batch size of 64, and using Adam optimization [15] coupled with cosine annealing schedule [16], which we configured to have a full cycle ever 2 epochs. We monitored the learning process by checking the losses obtained on 10% of data left for validation, and applied early stopping when

TABLE II
PERFORMANCE BREAKDOWN OF THE PROPOSED SFAT-NET ON THE SPOOFING ALGORITHMS OF THE ASVspoof LA TEST PARTITION

	all	A07	A08	A09	A10	A11	A12	A13	A14	A15	A16	A17	A18	A19
EER (%)	17.51	0.03	2.89	2.92	24.49	14.21	31.23	31.74	1.17	21.59	0.11	31.46	15.21	0.06
AUC (#)	0.900	0.999	0.995	0.995	0.811	0.924	0.736	0.729	0.999	0.847	0.999	0.734	0.913	0.999

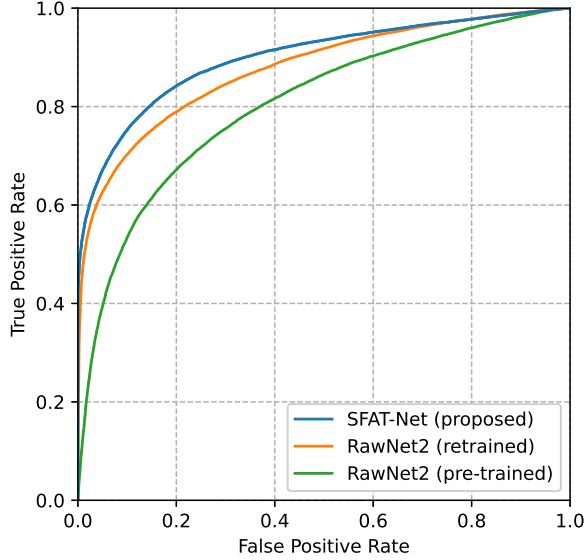


Fig. 5. ROC curves on the entire ASVspoof LA test partition

the validation reached a plateau, after about 50 epochs. The loss used for training was a plain sum without weighting of the three losses defined in Section II.

IV. EVALUATION

Due to the difficulties regarding data distribution within the available open datasets [4], comparing results with other state-of-the-art methods is challenging.

For the comparison, we relied on the RawNet2 [6] end-to-end network as a baseline for the state of the art. This model, initially proposed for speaker verification, was adapted by the ASVspoof committee as deepfake detection baseline for the 2019 challenge [3], and then updated and released to the public for the 2021 challenge [17].

Since the pre-trained model provided by the ASVspoof committee was trained on the initial content without silence trimming or normalization, we trained RawNet2 ourselves on pre-processed train and development partitions using the hyperparameters provided for the challenge, and considered this a second, alternative baseline.

For the sake of brevity, we will refer to our method by the acronym SFAT-Net in the following, a shortening of “speech formant analysis transformer network”.

A. Comparison with the ASVspoof 2021 baseline

The outcome of a first evaluation on the entire test partition of ASVspoof 2019 LA subset is depicted by the Receiver

TABLE III
PERFORMANCE ON THE ENTIRE ASVspoof LA TEST PARTITION

	RawNet2 (pre-trained)	RawNet2 (retrained)	SFAT-Net (proposed)
EER (%)	26.79	20.67	17.51
AUC (#)	0.803	0.877	0.900

Operating Characteristic (ROC) curves in Figure 5 and detailed in Table III, in which we reported both the Area Under the Curve (AUC) and EER we obtained – with the positive class corresponding to synthetic content.

Our proposed SFAT-Net obtained the lowest EER among the three architectures, equal to 17.51%. It is noteworthy that our results confirm that the lack of silence greatly penalizes the RawNet2 architecture: The pre-trained RawNet2 achieved a EER of 26.79%, whereas the retrained RawNet2 improved it to 20.67% – against an expected EER of 3.50%, according to the ASVspoof organizers [6].

Our proposed method is also better overall than the two baselines: The ROC curve obtained by the SFAT-Net is systematically higher than the ones obtained by RawNet2, and this tendency is also reflected in the AUC, which is the highest of the three and equal to 0.900.

B. Detailed outcome on ASVspoof 2019 LA

Synthesis detection performance on ASVspoof 2019 LA was reported to be strongly uneven in the accompanying summary paper of the challenge [3]: In particular, the attacks A10, A13 and A18 proved to severely degrade Automatic Speaker Verification (ASV) performance while at the same time being challenging to detect. In addition, the A17 attack proved to be the most difficult synthesis method to detect, although it still poses a low threat to ASV systems. Finally, attacks A16 and A19 are *known*, i.e. example synthetic content has been provided in the training set, albeit with different voices and utterances.

Therefore, in Table II, we reported EERs and AUCs of the ROC curves obtained for each separate synthesis algorithm in the test set. We can conclude that A12, A13, and A17 are indeed the most difficult algorithms to detect, which is consistent with the statistics reported in [3]. The AUC obtained for A18 is however already larger than 0.9, suggesting above-average performance.

Due to the heterogeneous composition of the ASVspoof test data, we were unable to isolate a pattern related to feature extraction or vocoding stage that could suggest the reason for the high error rates. To determine if the synthesis pipelines are

detectable at all with the proposed SFAT-Net, we performed training and cross-validation only for the worst performing algorithms: A10, A12, A13, A15, and A17. The outcome was an almost perfect detection, with an AUC of 0.999. Therefore, we suspect that the problem is indeed a very imbalanced split between train and test utterances of the ASVspoof LA partition, resulting in a strong domain mismatch between the two distributions.

V. SUMMARY AND OUTLOOK

In this paper, we proposed to address synthetic speech detection by means of a multi-task network with a strong feature bottleneck. The bottleneck is used to autoencode the input spectrogram, to predict the trajectory of the fundamental frequency f_0 , and only finally to classify the speech as synthetic or not synthetic. Therefore, the synthesis detection happens as byproduct of attending the energy of vocal formants, leading to a clear interpretation of which characteristics of the input signal influence the final output.

The proposed SFAT-Net was evaluated on the ASVspoof 2019 LA dataset, which was pre-processed to remove training/leading silence and to normalize the loudness according to the EBU recommendations. The model achieved better performance than RawNet2, a largely established anti-spoofing model which was tested both as provided by the ASVspoof challenge organizers, and after retraining on training content pre-processed as the evaluation one.

In future work, we plan to further evaluate performance on datasets with more balanced distribution, in order to understand the impact of the feature extraction and vocoding stages involved in both TTS and VC models. Furthermore, we plan to evaluate the robustness of the model with respect to lossy-encoding and the addition of background noise, without relying on data augmentation. Finally, we would like to tailor the architecture to the analysis of a single person of interest (POI), and include random masking of the input patches as proposed for vision autoencoders [18].

REFERENCES

- [1] N. M. Müller, K. Pizzi, and J. Williams, "Human perception of audio deepfakes," in *ACM International Workshop on Deepfake Detection for Audio Multimedia (DDAM)*, Lisboa, Portugal, 2022, pp. 85–91.
- [2] Europol, *Facing reality? Law enforcement and the challenge of deepfakes, an observatory report from the Europol Innovation Lab*. Publications Office of the European Union, 2022.
- [3] M. Todisco, X. Wang, V. Vestman, *et al.*, "Asvspoof 2019: Future horizons in spoofed and fake audio detection," in *ISCA Annual Conference of the International Speech Communication Association (INTERPEECH)*, Graz, Austria, 2019, pp. 1008–1012.
- [4] L. Cuccovillo, C. Papastergiopoulos, A. Vafeiadis, *et al.*, "Open challenges in synthetic speech detection," in *IEEE International Workshop on Information Forensics and Security (WIFS)*, Shanghai, China, 2022, pp. 1–6.
- [5] N. M. Müller, F. Ieckmann, P. Czempin, R. Canals, K. Böttinger, and J. Williams, "Speech is silver, silence is golden: What do ASVspoof-trained models really learn?" In *Automatic Speaker Verification and Spoofing Countermeasures Challenge (ASVspoof)*, 2021, pp. 55–60.
- [6] H. Tak, J. Patino, M. Todisco, A. Nautsch, N. Evans, and A. Larcher, "End-to-end anti-spoofing with RawNet2," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Toronto, ON, Canada, 2021, pp. 6369–6373.
- [7] N. M. Müller, P. Czempin, F. Dieckmann, A. Froghyar, and K. Böttinger, "Does audio deepfake detection generalize?" In *ISCA Annual Conference of the International Speech Communication Association (INTERPEECH)*, Incheon, Korea, 2022, pp. 2783–2787.
- [8] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *IEEE International Conference on Computer Vision (ICCV)*, Venice, Italy, 2017, pp. 618–626.
- [9] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *International Conference on Neural Information Processing Systems (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 5998–6008.
- [10] A. Dosovitskiy, L. Beyer, A. Kolesnikov, *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations (ICLR)*, Vienna, Austria, 2021.
- [11] M. Mauch and S. Dixon, "PYIN: A fundamental frequency estimator using probabilistic threshold distributions," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Florence, Italy, 2014, pp. 659–663.
- [12] X. Wang, J. Yamagishi, M. Todisco, *et al.*, "ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech," *Computer Speech & Language*, vol. 64, 2020.
- [13] F. Jia, S. Majumdar, and B. Ginsburg, "MarbleNet: Deep 1D time-channel separable convolutional neural network for voice activity detection," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Toronto, ON, Canada, 2021, pp. 6818–6822.
- [14] EBU, *R128-2020: Loudness Normalization and Permitted Maximum Level of Audio Signals*. 2020.
- [15] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [16] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations (ICLR)*, Toulon, France, 2017.
- [17] X. Liu, X. Wang, M. Sahidullah, *et al.*, "ASVspoof 2021: Towards spoofed and deepfake speech detection in the wild," *IEEE/ACM Transactions on Audio, Speech, and Language Processing (TASLP)*, vol. 31, pp. 2507–2522, 2023.
- [18] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. B. Girshick, "Masked autoencoders are scalable vision learners," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, New Orleans, LA, USA, 2022, pp. 15 979–15 988.